



Universidad Mariano Gálvez de Guatemala.

REQUERIMIENTOS ÁGILES, BDD, UX/UI Y DESARROLLO
INICIAL DE API

CURSO: ANÁLISIS DE SISTEMAS II

- DYLAN ALBERTO ROLDAN PEREZ	9989-22-5223
FELIX FRANCISCO TOMA RODRIGUEZ	99989-12-10925
DULCE CLAUDIA MICHELLE RAMIREZ RAC	9989-23-15707
ANTHONY DE LEON	9989-21-5704

INTRODUCCIÓN	3
1. REQUERIMIENTOS ÁGILES Y PRODUCT BACKLOG	4
1.1. Descripción de Épicas y Mínimo Producto Viable (MVP)	4
1.2. Criterios de Priorización y Estimación (Story Points)	5
1.3. Tabla Consolidada de Historias de Usuario	6
Tablero Ágil:	8
2. ESPECIFICACIONES BDD (CRITERIOS DE ACEPTACIÓN)	9
2.1. Cobertura de Escenarios Gherkin	9
3. DISEÑO DE EXPERIENCIA DE USUARIO (UX/UI)	11
3.1. Mapa de Navegación del Sistema	12
3.2. Mapeo de Pantallas vs. Endpoints OpenAPI	13
3.3. Reporte de Evaluación de Usabilidad y Accesibilidad (WCAG 2.1)	14
4. ARQUITECTURA DEL CASCARÓN BACKEND (BOILERPLATE)	18
4.1. Explicación de la Estructura de Capas Elegida	18
4.2. Configuración de Conexión a Base de Datos	19
4.3. Diagrama de Secuencia del Endpoint Principal	21
5. IMPLEMENTACIÓN Y PRUEBA DE ENDPOINTS INICIALES	23
5.1. Detalle del Endpoint de Lectura (GET) Refactorizado	23
5.2. Detalle del Endpoint de Escritura (POST) Refactorizado	25
5.3. Evidencias de Ejecución (Postman/Insomnia)	28
Evidencia 1 — Inicio de sesión	28
Evidencia 2 — Consulta de contactos	29
Evidencia 3 — Creación correcta de contacto	30
Evidencia 4 — Validación de datos incorrectos	31
Colección de pruebas	31
6. MATRIZ DE TRAZABILIDAD ÁGIL	32
Observación de implementación	33
8. CONCLUSIONES Y LECCIONES APRENDIDAS	33
Conclusión 1	33
Conclusión 2	34
Conclusión 3	34
Conclusión 4	34
Conclusión 5	34
9. REFERENCIAS	34



INTRODUCCIÓN

La fase de análisis de requerimientos y prototipado tiene como propósito organizar la evolución del módulo CRM a partir de la arquitectura, el modelo de datos y el contrato OpenAPI definidos en el Entregable 1. Se busca transformar las necesidades del sistema en historias de usuario priorizadas y diseñar las pantallas que permitirán consultar y registrar información de manera clara. El módulo comprende la gestión de contactos generales, clientes, proveedores y empleados, junto con la administración de usuarios y permisos.

El desarrollo se realizará de forma incremental, mediante entregas pequeñas que puedan verificarse antes de ampliar el alcance. Los objetivos de este ciclo son delimitar el Mínimo Producto Viable (MVP), establecer un backlog priorizado, definir criterios de aceptación, elaborar un prototipo navegable y refactorizar las primeras operaciones de la API conservando las reglas de negocio del módulo.

La versión revisada utiliza controladores, modelos y vistas de Laravel, con autenticación mediante sesiones. En esta fase se propone exponer las primeras operaciones mediante una API con respuestas JSON y autenticación JWT, distribuyendo las responsabilidades entre controladores, servicios, modelos o repositorios y DTOs.

El primer incremento contempla los siguientes endpoints del contrato:

Método	Endpoint	Alcance inicial
POST	/api/v1/auth/login	Validar credenciales de una cuenta activa y emitir el token JWT necesario para acceder a los recursos protegidos.
GET	/api/v1/contactos	Consultar los contactos permitidos para el usuario, con búsqueda, filtros y paginación.
POST	/api/v1/contactos	Registrar un contacto general, cliente, proveedor o empleado, validar sus datos y guardar la información en la base de datos.

Las operaciones de consulta y registro deberán responder con los DTOs y códigos HTTP establecidos en el contrato: 200 OK para las consultas correctas y 201 Created para los registros correctos. La autenticación es una dependencia del flujo, por lo que se incluye además del mínimo de un endpoint GET y uno POST solicitado. La consulta individual, actualización y desactivación se planifican para los siguientes incrementos.

1. REQUERIMIENTOS ÁGILES Y PRODUCT BACKLOG

1.1. Descripción de Épicas y Mínimo Producto Viable (MVP)

Las épicas agrupan las funcionalidades según su responsabilidad dentro del negocio. Se consideran dos roles: usuario estándar, que trabaja con los contactos asignados a su cuenta, y administrador, que puede supervisar todos los contactos y administrar usuarios.

ID	Épica	Descripción	Historias asociadas
EP-01	Acceso y seguridad	Permitir el acceso autenticado de cuentas activas y aplicar los permisos del usuario.	HU-01; permisos transversales en las demás historias.
EP-02	Gestión de contactos	Consultar, registrar, buscar, actualizar y desactivar contactos según los permisos establecidos.	HU-02, HU-03, HU-04, HU-08, HU-09, HU-10 y HU-11.
EP-03	Clasificación de contactos	Registrar la información específica de clientes, proveedores y empleados.	HU-05, HU-06 y HU-07.
EP-04	Administración de usuarios	Crear cuentas y gestionar sus datos, roles y estado de acceso.	HU-12 y HU-13.
EP-05	Resumen del CRM	Presentar cantidades por tipo y estado de contacto, además de los registros recientes.	HU-14.

MVP del ciclo de refactorización. Se propone un flujo que permita a una cuenta activa autenticarse, consultar los contactos autorizados y registrar nuevos contactos de cualquiera de los cuatro tipos.

Incluirá validaciones, control de permisos, búsqueda, filtros, paginación y persistencia en la base de datos. Las pantallas de acceso, listado y registro se representarán en el prototipo navegable.

Este alcance corresponde a las historias **HU-01 a HU-08**, con una estimación inicial de 27 **Story Points**. El MVP utilizará cuentas ya disponibles en el módulo; la refactorización de la administración de usuarios se abordará después. Los tres endpoints indicados en la introducción permitirán demostrar el flujo mediante un cliente HTTP.

El backlog también conserva funcionalidades del producto para los siguientes incrementos. Las historias de administración de usuarios y del panel de resumen reflejan funciones existentes en la aplicación web. Su exposición mediante API requerirá ampliar el contrato OpenAPI antes de implementarlas por esa vía.

1.2. Criterios de Priorización y Estimación (Story Points)

Se utilizará MoSCoW para ordenar las historias según su valor para el negocio, dependencia con otras funciones, impacto en la seguridad y necesidad para completar el flujo de consulta y registro.

Categoría	Prioridad	Criterio aplicado
Must Have	Alta	Funcionalidades indispensables para el MVP de este ciclo: acceso, permisos, consulta y registro de contactos.
Should Have	Media	Funcionalidades importantes para completar la operación del CRM, programadas después del flujo inicial.
Could Have	Baja	Funcionalidades complementarias que aportan comodidad y pueden abordarse cuando exista capacidad disponible.
Won't Have en este ciclo	Fuera del ciclo	Ampliaciones ajenas al alcance acordado. No forman parte del compromiso de este incremento.

Dentro de una misma prioridad se atenderán primero las dependencias: autenticación antes del acceso a contactos y registro de datos generales antes de incorporar las particularidades de cada clasificación.

El orden de las filas de la tabla representa el orden inicial de atención del backlog.

La estimación utilizará la secuencia de Fibonacci 1, 2, 3, 5, 8 y 13. Los puntos expresan esfuerzo relativo, complejidad e incertidumbre; no equivalen directamente a horas ni días. Como referencia, 2 puntos corresponden a un ajuste acotado sobre una función existente; 3, a una adaptación con validaciones o permisos; y 5, a un flujo que combina varias responsabilidades e integración.

Los valores son una propuesta inicial para revisión del equipo y estiman el trabajo pendiente de adaptación, integración y verificación sobre el módulo existente. Las tareas técnicas compartidas se registrarán una sola vez y se vincularán a las historias que correspondan. El backlog completo suma **50 Story Points** el primer incremento comprende los **27 puntos** de las historias Must Have. Estas cifras no constituyen un compromiso de duración sin conocer la capacidad del equipo.

1.3. Tabla Consolidada de Historias de Usuario

En la tabla, «usuario autenticado» comprende a los roles estándar y administrador, respetando los permisos de cada uno.

ID	Historia de Usuario	Épica	Prioridad	Story Points
HU-01	Como usuario con cuenta activa, quiero iniciar sesión con mi correo y contraseña para acceder a las operaciones autorizadas del CRM.	EP-01	Alta (Must)	5
HU-02	Como usuario estándar, quiero consultar mis contactos en un listado paginado para organizar la información de mi cartera.	EP-02	Alta (Must)	3
HU-03	Como administrador, quiero consultar los contactos de todos los usuarios para supervisar la información registrada en el CRM.	EP-02	Alta (Must)	2
HU-04	Como usuario autenticado, quiero registrar un contacto general con sus datos básicos para centralizar su información de comunicación.	EP-02	Alta (Must)	5
HU-05	Como usuario autenticado, quiero registrar un contacto como cliente con un código único para identificarlo dentro de la cartera de clientes.	EP-03	Alta (Must)	3
HU-06	Como usuario autenticado, quiero registrar un contacto como proveedor con una descripción para identificar el tipo de suministro o servicio que ofrece.	EP-03	Alta (Must)	3
HU-07	Como usuario autenticado, quiero registrar un contacto como empleado con su puesto y departamento para identificar su función dentro de la organización.	EP-03	Alta (Must)	3



HU-08	Como usuario autenticado, quiero buscar y filtrar los contactos autorizados por sus datos, tipo y estado para encontrar la información que necesito.	EP-02	Alta (Must)	3
HU-09	Como usuario autenticado, quiero consultar el detalle de un contacto autorizado para revisar su información general y específica.	EP-02	Media (Should)	3
HU-10	Como usuario autenticado, quiero actualizar los datos y la clasificación de un contacto autorizado para mantener su información vigente.	EP-02	Media (Should)	5
HU-11	Como administrador, quiero desactivar un contacto sin eliminar sus datos para retirarlo de la operación activa y conservar su registro.	EP-02	Media (Should)	2
HU-12	Como administrador, quiero crear cuentas de usuario con un rol definido para habilitar el acceso del personal autorizado al CRM.	EP-04	Media (Should)	5
HU-13	Como administrador, quiero actualizar los datos, el rol y el estado de las cuentas para mantener los accesos conforme a las responsabilidades del personal.	EP-04	Media (Should)	5
HU-14	Como usuario autenticado, quiero visualizar un resumen de mis contactos autorizados por tipo y estado, junto con los registros recientes, para conocer la situación de la información que gestiono.	EP-05	Baja (Could)	3

Reglas de negocio asociadas. Se incluyen para mantener la relación entre las historias y las restricciones del módulo exigida por la rúbrica.



ID	Regla de negocio	Historias relacionadas
RN-01	El acceso requiere credenciales válidas y una cuenta activa. Las operaciones protegidas de la API requieren un token válido y deberán comprobar que la cuenta continúa activa.	HU-01 a HU-14, en las operaciones que se expongan mediante API.
RN-02	El usuario estándar solo puede consultar y actualizar contactos asociados a su cuenta. El administrador puede consultar y actualizar todos. Los listados, búsquedas y resúmenes deben respetar ese alcance.	HU-02, HU-03, HU-08, HU-09, HU-10 y HU-14.
RN-03	Todo contacto requiere tipo, nombre, apellido y teléfono. Correo, dirección y empresa son opcionales, pero deben cumplir las validaciones cuando se proporcionen.	HU-04 a HU-07 y HU-10.
RN-04	El teléfono debe contener entre 8 y 15 dígitos y no repetirse en otro contacto. El correo, cuando se registre, debe tener formato válido y ser único entre los contactos. Al actualizar se excluye el propio registro de la comprobación de unicidad.	HU-04 a HU-07 y HU-10.
RN-05	Cada contacto tiene un solo tipo: general, cliente, proveedor o empleado. Un cliente requiere un código entero positivo y único; un proveedor, una descripción; y un empleado, puesto y departamento.	HU-04 a HU-07 y HU-10.
RN-06	En el primer incremento de la API, el nuevo contacto se asociará a la cuenta autenticada y quedará activo. Sus datos generales y de clasificación deberán guardarse como una sola operación: si una parte falla, no se conservará un registro incompleto.	HU-04 a HU-07.
RN-07	Solo el administrador puede desactivar contactos. La desactivación cambia su estado y conserva los datos; no elimina físicamente el contacto.	HU-11.
RN-08	Solo el administrador puede crear o modificar cuentas y asignar los roles estándar o administrador. El correo de usuario debe ser válido y único. Las nuevas contraseñas deben tener al menos ocho caracteres, incluir letras y números, y almacenarse mediante hash.	HU-12 y HU-13.
RN-09	No se permite desactivar la propia cuenta ni desactivar o quitar el rol al último administrador activo.	HU-13.

Relación con el contrato y la implementación.

Para la API se respetarán los parámetros buscar, tipo, estado, page y per_page definidos en OpenAPI. La paginación tendrá 10 registros por defecto y un máximo de 100 por página. La adaptación deberá traducir esos parámetros al modelo existente, ya que la interfaz web utiliza nombres o valores diferentes en algunos filtros. La asignación de contactos a otro usuario que existe en la interfaz administrativa queda fuera del POST inicial porque el DTO de creación actual no incluye id_usuario como dato de entrada

Tablero Ágil: [JIRA LINK](#)

2. ESPECIFICACIONES BDD (CRITERIOS DE ACEPTACIÓN)

2.1. Cobertura de Escenarios Gherkin

Las especificaciones BDD del módulo CRM se documentaron utilizando sintaxis **Gherkin en español**, mediante las palabras clave Característica, Escenario, Dado, Cuando, Entonces y Y.

Los escenarios fueron organizados en archivos .feature según las principales responsabilidades funcionales del sistema:

- autenticacion.feature: acceso y autenticación.
- gestion_contactos.feature: consulta, búsqueda, actualización y desactivación de contactos.
- clasificacion_contactos.feature: registro de clientes, proveedores y empleados.
- administracion_usuarios.feature: creación y modificación de cuentas.
- resumen_crm.feature: visualización del resumen general del CRM.

Cada historia de usuario dispone de al menos un **escenario principal o camino feliz**, encargado de validar el comportamiento esperado cuando se cumplen las condiciones necesarias.

También se incluyen **escenarios de excepción y alternos**, considerando situaciones como credenciales inválidas, usuarios sin permisos, datos obligatorios faltantes, valores duplicados, recursos inexistentes y restricciones propias de las reglas de negocio.

Los escenarios mantienen relación con las reglas RN-01 a RN-09 definidas en el Product Backlog. Por ejemplo, se valida el alcance de los contactos según el usuario autenticado, la unicidad de teléfonos y correos, los datos específicos según el tipo de contacto y las restricciones aplicables a las cuentas administrativas. backlog

Para HU-12, HU-13 y HU-14 se especifica el comportamiento funcional sin definir nuevas rutas HTTP, debido a que la exposición de esas funciones mediante API pertenece a incrementos posteriores y requiere ampliar previamente el contrato OpenAPI.

Historia de Usuario	Archivo .feature	Escenarios Felices	Escenarios de Error
HU-01	docs/agile/features/autenticacion.feature	1	2
HU-02	docs/agile/features/gestion_contactos.feature	1	2
HU-03	docs/agile/features/gestion_contactos.feature	1	2
HU-04	docs/agile/features/gestion_contactos.feature	1	3
HU-05	docs/agile/features/clasificacion_contactos.feature	1	2
HU-06	docs/agile/features/clasificacion_contactos.feature	1	2
HU-07	docs/agile/features/clasificacion_contactos.feature	1	2
HU-08	docs/agile/features/gestion_contactos.feature	1	2
HU-09	docs/agile/features/gestion_contactos.feature	1	3
HU-10	docs/agile/features/gestion_contactos.feature	1	3
HU-11	docs/agile/features/gestion_contactos.feature	1	3
HU-12	docs/agile/features/administracion_usuarios.feature	1	3
HU-13	docs/agile/features/administracion_usuarios.feature	1	4
HU-14	docs/agile/features/resumen_crm.feature	2	1

[LINK Archivos](#)

3. DISEÑO DE EXPERIENCIA DE USUARIO (UX/UI)

El diseño de experiencia de usuario del módulo CRM se plantea bajo un enfoque **Responsive Web**, procurando que las pantallas puedan utilizarse tanto desde equipos de escritorio como desde dispositivos móviles.

El prototipo interactivo representa los principales flujos definidos en las historias de usuario y en el contrato OpenAPI. Se priorizan las operaciones de autenticación y gestión de contactos incluidas en el MVP, mientras que las funciones de administración de usuarios y resumen del CRM se consideran pantallas complementarias para incrementos posteriores.

Para mantener consistencia visual se utilizarán componentes reutilizables para botones, campos de texto, selectores, tarjetas, tablas, mensajes de validación, ventanas de confirmación y navegación. En dispositivos con menor ancho, las tablas podrán transformarse en tarjetas o listados verticales con el objetivo de conservar legibilidad y facilidad de interacción.

El prototipo deberá contemplar como mínimo las siguientes pantallas:

- Inicio de sesión.
- Listado de contactos.
- Formulario para registrar contacto.
- Detalle de contacto.
- Edición de contacto.
- Confirmación de desactivación.
- Resumen del CRM.
- Administración de usuarios, como función prevista para un incremento posterior.

Los diseños se desarrollarán inicialmente mediante **wireframes de baja fidelidad**, enfocados en distribución y navegación, y posteriormente como **mockups de mayor fidelidad** que incorporen tipografías, colores, espaciado, estados de controles y comportamiento responsive.

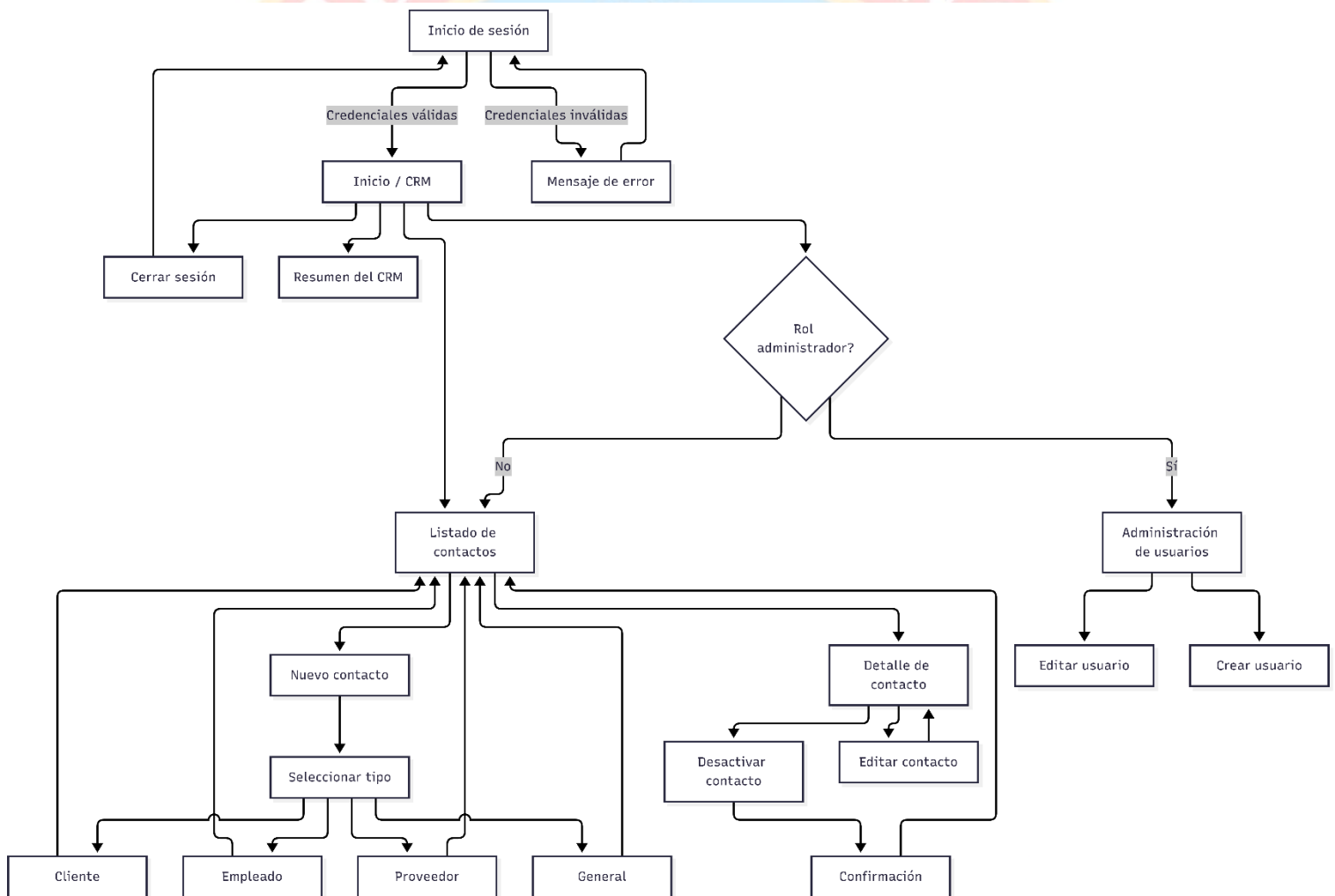
3.1. Mapa de Navegación del Sistema

El mapa de navegación representa las principales transiciones disponibles dentro del módulo CRM.

El flujo comienza en la pantalla de inicio de sesión. Después de validar las credenciales, el usuario accede al sistema y puede consultar el listado de contactos. Desde esta pantalla puede realizar búsquedas y filtros, registrar un nuevo contacto o consultar la información de uno existente.

Desde el detalle de un contacto se permite acceder a la edición y, cuando el rol posee autorización, a la opción de desactivación. Las funciones administrativas y de resumen se muestran según el alcance definido para cada rol.

La navegación busca reducir la cantidad de pasos necesarios para ejecutar las operaciones principales del sistema y mantener disponibles las acciones relacionadas con el contexto actual.



El mapa distingue las operaciones accesibles para un usuario estándar de aquellas reservadas para administración. La aplicación debe respetar el alcance de los contactos asociados al usuario

autenticado, mientras que el administrador puede consultar todos los contactos definidos por las reglas del backlog.

3.2. Mapeo de Pantallas vs. Endpoints OpenAPI

La interfaz se relaciona directamente con las operaciones definidas en el contrato OpenAPI del módulo CRM.

El contrato contiene POST /auth/login, GET /contactos, POST /contactos, GET /contactos/{id}, PUT /contactos/{id} y DELETE /contactos/{id}

Pantalla / Acción	Historia relacionada	Endpoint OpenAPI	Método	Comportamiento visual
Inicio de sesión	HU-01	/api/v1/auth/login	POST	Envía correo y contraseña. Al recibir respuesta correcta permite ingresar al CRM.
Listado de contactos	HU-02, HU-03	/api/v1/contactos	GET	Presenta los contactos autorizados de forma paginada.
Buscar y filtrar	HU-08	/api/v1/contactos	GET	Utiliza parámetros de búsqueda, tipo, estado y paginación.
Registrar contacto	HU-04 a HU-07	/api/v1/contactos	POST	Envía datos generales y campos específicos según el tipo de contacto.
Detalle de contacto	HU-09	/api/v1/contactos/{id}	GET	Muestra los datos generales y específicos del contacto seleccionado.
Editar contacto	HU-10	/api/v1/contactos/{id}	PUT	Envía los datos modificados y actualiza la información presentada.
Desactivar contacto	HU-11	/api/v1/contactos/{id}	DELETE	Solicita confirmación y cambia el contacto a estado inactivo.
Administración de usuarios	HU-12, HU-13	No definido actualmente	—	Funcionalidad prevista para ampliación futura del contrato.
Resumen del CRM	HU-14	No definido actualmente	—	Funcionalidad prevista para ampliación futura del contrato.

Pantalla 1 - Login

→ POST /api/v1/auth/login

→ 200 / 401

→ Contactos / Error

Pantalla 2 - Contactos

→ GET /api/v1/contactos

→ 200

→ Listado actualizado

Pantalla 3 - Nuevo Contacto

→ POST /api/v1/contactos

→ 201 / 422

→ Contactos / Validaciones

Pantalla 2 - Contactos → Ver

→ GET /api/v1/contactos/{id}

→ 200 / 404

→ Detalle / No encontrado

Pantalla 5 - Editar

→ PUT /api/v1/contactos/{id}

→ 200 / 422 / 404

→ Detalle actualizado / Error

Pantalla 6 - Confirmar Desactivación

→ DELETE /api/v1/contactos/{id}

→ 200 / 404

→ Contactos con registro inactivo

3.3. Reporte de Evaluación de Usabilidad y Accesibilidad (WCAG 2.1)

La evaluación del prototipo se orienta a verificar que las pantallas sean comprensibles, consistentes, operables y adaptables a diferentes tamaños de dispositivo.

Como referencia de accesibilidad se utilizan criterios de WCAG 2.1, principalmente aquellos relacionados con contraste, legibilidad, escalabilidad, navegación mediante teclado y tamaño de elementos interactivos.

Aspecto evaluado	Criterio aplicado	Verificación propuesta	Resultado
Contraste de texto normal	Mínimo 4.5:1	Verificar texto, etiquetas y botones contra el fondo	Cumple / Por verificar en Figma
Contraste de texto grande	Mínimo 3:1	Verificar títulos y textos de gran tamaño	Cumple / Por verificar
Tipografía	Texto claramente legible	Tamaño base recomendado de 16 px y jerarquía visual consistente	Cumple / Por verificar
Escalado	Hasta 200 % sin pérdida de contenido	Simular zoom y diseño responsive	Cumple / Por verificar
Áreas de toque	Objetivo de 44 × 44 CSS px cuando sea posible	Revisar botones, íconos y controles interactivos	Cumple / Por verificar
Formularios	Etiquetas visibles asociadas a los controles	Revisar login y formularios de contactos	Cumple / Por verificar
Errores	No comunicar errores únicamente mediante color	Agregar texto e ícono además del color	Cumple / Por verificar
Navegación	Orden lógico y consistente	Revisar menú, botones atrás y acciones principales	Cumple / Por verificar
Foco	Elemento enfocado visualmente identificable	Validar navegación por teclado en implementación	Pendiente de implementación
Diseño responsive	Contenido adaptable a móvil y escritorio	Verificar mockups en varios anchos	Cumple / Por verificar

WCAG 2.1 nivel AA establece una relación mínima de contraste de 4.5:1 para texto normal y 3:1 para texto grande. W3 También establece que el texto debe poder ampliarse hasta 200 % sin pérdida de contenido o funcionalidad. W3

Para las áreas táctiles se adopta como objetivo de diseño 44 × 44 CSS px. Este tamaño corresponde al criterio 2.5.5 de WCAG 2.1, que es de nivel AAA; por tanto, se utiliza aquí como buena práctica adicional y no como requisito mínimo AA. W3

Contraste

El prototipo debe procurar que:

- El texto principal mantenga suficiente contraste respecto al fondo.
- Los textos secundarios no utilicen tonos excesivamente claros.
- Los botones primarios conserven texto claramente visible.
- Los estados de error no dependan únicamente del color rojo.
- Los estados activo/inactivo incluyan texto, icono o etiqueta además del color.

La validación final puede realizarse utilizando herramientas de contraste disponibles para Figma o verificadores compatibles con WCAG.

Tipografía

La interfaz utilizará una tipografía sans-serif de fácil lectura y una jerarquía consistente.

- Título principal: 28–32 px
- Título de sección: 20–24 px
- Texto normal: 16 px
- Etiquetas: 14–16 px
- Texto auxiliar: 14 px

Áreas de toque

En la vista móvil, las principales acciones deben disponer de suficiente superficie para facilitar la interacción táctil.

Se aplicará especialmente a:

- Botón Iniciar sesión.
- Botón Nuevo contacto.
- Botón Guardar.
- Botón Editar.
- Botón Desactivar.
- Botones de filtros.
- Navegación.
- Elementos de paginación.

También debe existir separación suficiente entre acciones críticas; por ejemplo, **Editar** y **Desactivar** no deberían ubicarse tan próximas que sea fácil seleccionar la opción incorrecta.

Formularios

Los formularios deberán mostrar claramente:

- Nombre del campo.
- Campos obligatorios.
- Formato esperado cuando sea necesario.
- Mensaje específico cuando exista un error.
- Estado de éxito después de guardar información.

Evaluación de usabilidad

Para verificar el prototipo se proponen las siguientes tareas de prueba:

Prueba	Tarea	Resultado esperado
U01	Iniciar sesión correctamente	El usuario identifica fácilmente los campos y accede al sistema
U02	Buscar un contacto	El usuario encuentra rápidamente la barra de búsqueda
U03	Filtrar contactos por tipo	El filtro es identificable y el resultado refleja la selección
U04	Crear un nuevo contacto	El usuario completa el flujo sin confundir los campos obligatorios
U05	Registrar un cliente	La interfaz muestra el campo adicional de código de cliente
U06	Consultar detalle	La acción para abrir un contacto es fácil de identificar
U07	Editar información	El usuario reconoce qué campos puede modificar
U08	Desactivar contacto	El sistema solicita confirmación antes de ejecutar una acción sensible
U09	Utilizar el prototipo en móvil	La interfaz se mantiene legible sin desplazamiento horizontal innecesario
U10	Provocar un error de formulario	El mensaje explica claramente qué debe corregirse

Conclusión de evaluación

El diseño propuesto busca mantener una navegación sencilla, reducir pasos innecesarios y presentar las operaciones principales del CRM mediante patrones visuales consistentes.

La aplicación de criterios WCAG 2.1 permitirá mejorar la legibilidad y operabilidad de la interfaz, especialmente mediante un contraste adecuado, tipografías legibles, mensajes de error explícitos y controles con áreas de interacción suficientes.

La evaluación definitiva deberá realizarse sobre los mockups terminados y posteriormente sobre la implementación funcional, debido a que determinados aspectos, como navegación mediante

teclado, foco visible y comportamiento real al ampliar la interfaz, no pueden verificarse completamente mediante un prototipo estático.

Vinculo al prototipo Figma: [LINK](#)

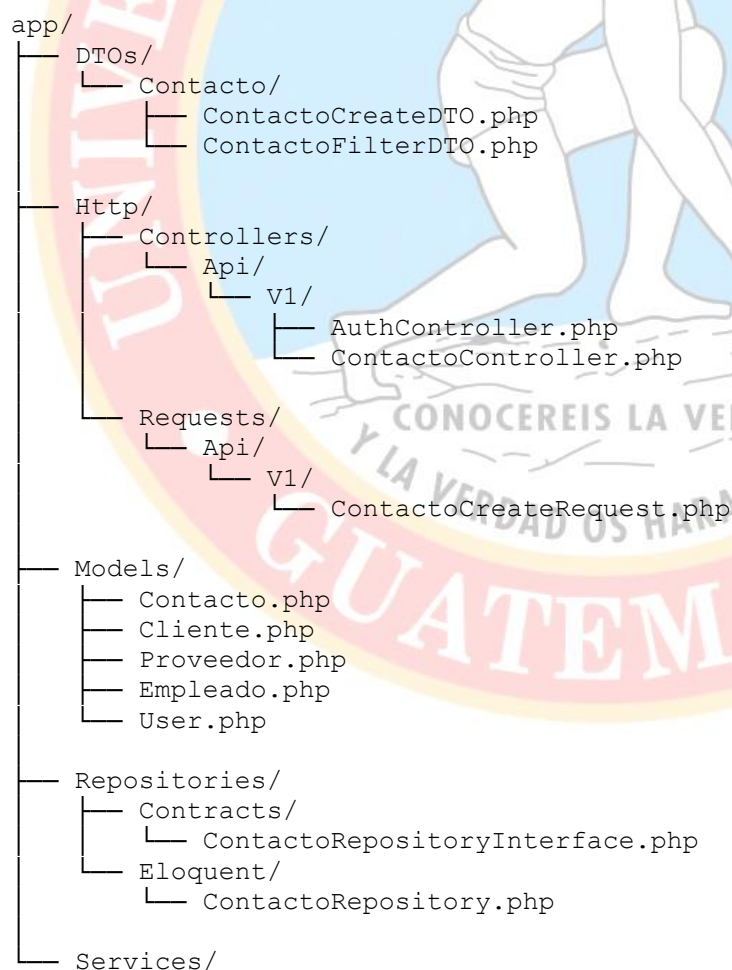
4. ARQUITECTURA DEL CASCARÓN BACKEND (BOILERPLATE)

4.1. Explicación de la Estructura de Capas Elegida

Para el backend del módulo CRM se implementó una arquitectura organizada por capas sobre **Laravel**, con el propósito de separar las responsabilidades de recepción de solicitudes HTTP, validación, lógica de negocio y acceso a datos.

La estructura desarrollada está compuesta por **Controladores, Requests, DTOs, Servicios, Repositorios y Modelos Eloquent**. Esta separación reduce el acoplamiento entre componentes y facilita el mantenimiento, las pruebas y futuras ampliaciones de la API.

La estructura principal implementada es la siguiente:



└─ ContactoService.php

Las responsabilidades se distribuyen de la siguiente manera:

Capa	Responsabilidad
Controller	Recibe las solicitudes HTTP y devuelve las respuestas JSON.
Request	Valida los datos recibidos antes de ejecutar la lógica de negocio.
DTO	Transporta los datos entre las distintas capas del sistema.
Service	Coordina la lógica de negocio de los contactos.
Repository	Centraliza las operaciones de lectura y escritura en la base de datos.
Model	Representa las tablas de MySQL mediante Eloquent ORM.

El flujo general implementado es:

Cliente HTTP
↓
Middleware JWT
↓
Controller
↓
Request / DTO
↓
Service
↓
Repository
↓
Eloquent
↓
MySQL

Esta estructura permite que el controlador no realice consultas directamente sobre la base de datos. En su lugar, delega la operación al servicio, y este utiliza el repositorio correspondiente.

También se implementó una interfaz denominada `ContactoRepositoryInterface`, cuya implementación concreta es `ContactoRepository`. La asociación entre ambos componentes se realiza mediante el contenedor de dependencias de Laravel en `AppServiceProvider`.

4.2. Configuración de Conexión a Base de Datos

Para la persistencia del módulo CRM se utiliza **MySQL** como sistema gestor de base de datos relacional.

La aplicación Laravel obtiene la configuración de conexión mediante variables de entorno definidas en el archivo `.env`.

La configuración utilizada tiene la siguiente estructura:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=modulocrmLaravel
DB_USERNAME=root
DB_PASSWORD=
```

Laravel procesa estos valores mediante:

```
config/database.php
```

y utiliza **Eloquent ORM** para realizar las operaciones de persistencia.

La conexión fue comprobada mediante:

```
php artisan migrate:status
```

El comando confirmó que las migraciones principales del CRM se encontraban ejecutadas correctamente, incluyendo las tablas:

```
usuarios
contactos
clientes
proveedores
empleados
```

También se verificó directamente la comunicación entre el modelo y la base de datos utilizando Laravel Tinker:

```
App\Models\Contacto::count();
```

y:

```
App\Models\Contacto::first();
```

Estas pruebas permitieron comprobar que Laravel podía consultar registros almacenados en MySQL correctamente.

La relación entre aplicación y base de datos puede representarse como:

```
Laravel
↓
Eloquent ORM
↓
Modelos
↓
MySQL
↓
modulocrmLaravel
```

Para las operaciones de creación de contactos también se utilizó:

```
DB::transaction()
```

Esto permite que los datos generales del contacto y su clasificación específica se almacenen como una única operación.

Por ejemplo:

Contacto cliente

```
INSERT contactos
```

```
↓  
INSERT clientes
```

```
↓  
COMMIT
```

Si ocurre un error durante el segundo `INSERT`, Laravel ejecuta un rollback para evitar que quede un contacto incompleto.

4.3. Diagrama de Secuencia del Endpoint Principal

Para representar el funcionamiento interno de la API se seleccionó el endpoint:

```
POST /api/v1/contactos
```

Este endpoint permite observar todas las capas implementadas en el backend, desde la solicitud realizada por el cliente hasta la persistencia de la información en MySQL.

La secuencia inicia cuando el cliente HTTP envía la solicitud acompañada de un **Bearer Token JWT**.

El middleware valida el token y, si la autenticación es correcta, la solicitud pasa a `ContactoCreateRequest`, donde se verifican los datos enviados.

Posteriormente, `ContactoController` crea un objeto `ContactoCreateDTO` y solicita a `ContactoService` el registro del contacto.

El servicio delega la persistencia a `ContactoRepository`, donde se inicia una transacción de base de datos.

Primero se registra la información general en:

```
contactos
```

Luego, dependiendo de `tipo_contacto`, también puede almacenarse información en:

clientes
proveedores
empleados

Finalmente, si todas las operaciones se ejecutan correctamente, se confirma la transacción y el controlador devuelve:

201 Created

El flujo principal puede resumirse como:

Postman
↓
JWT Middleware
↓
ContactoCreateRequest
↓
ContactoController
↓
ContactoCreateDTO
↓
ContactoService
↓
ContactoRepository
↓
Modelos Eloquent
↓
MySQL
↓
201 Created

El diagrama también contempla respuestas alternativas:

Token inválido
→ 401 Unauthorized

Datos inválidos
→ 422 Unprocessable Entity

Creación correcta
→ 201 Created

Vínculo al Diagrama de Secuencia:

[LINK](#)

5. IMPLEMENTACIÓN Y PRUEBA DE ENDPOINTS INICIALES

5.1. Detalle del Endpoint de Lectura (GET) Refactorizado

Se implementó el endpoint:

```
GET /api/v1/contactos
```

Su propósito es consultar los contactos almacenados en la base de datos, permitiendo aplicar filtros y paginación.

El endpoint se encuentra protegido mediante autenticación **Bearer JWT**, por lo que el cliente debe enviar un token válido en el encabezado `Authorization`.

La solicitud puede incluir los siguientes parámetros opcionales:

Parámetro	Descripción
buscar	Permite buscar por nombre, apellido, correo, teléfono o empresa.
tipo	Filtra por general, cliente, proveedor o empleado.
estado	Filtra por activo o inactivo.
page	Indica la página solicitada.
per_page	Define la cantidad de registros por página.

Ejemplo:

```
GET /api/v1/contactos?tipo=cliente&estado=activo&page=1&per_page=10
```

La implementación sigue la siguiente secuencia:

```
GET /api/v1/contactos
↓
Middleware JWT
↓
ContactoController@index
↓
ContactoFilterDTO
↓
ContactoService
↓
ContactoRepository
↓
Contacto Model
↓
MySQL
```

El controlador valida los parámetros recibidos y los encapsula en un objeto

ContactoFilterDTO.

Posteriormente, `ContactoService` delega la consulta a `ContactoRepository`, el cual construye la consulta utilizando Eloquent ORM.

El repositorio aplica dinámicamente los filtros solicitados antes de ejecutar la consulta paginada.

Una respuesta exitosa utiliza el código:

200 OK

Ejemplo de respuesta:

```
{
  "success": true,
  "message": "Contactos consultados correctamente.",
  "data": [
    {
      "id_contacto": 4,
      "id_usuario": 1,
      "tipo_contacto": "cliente",
      "nombre": "Carlos",
      "apellido": "Lopez",
      "correo": "carlos.api@example.com",
      "numero_telefonico": "55554321",
      "direccion": "Ciudad de Guatemala",
      "empresa": "Comercial API",
      "estado_activo": true
    }
  ],
  "meta": {
    "pagina_actual": 1,
    "ultima_pagina": 1,
    "por_pagina": 10,
    "total": 4
  }
}
```

En caso de que no se proporcione un token válido, el endpoint responde:

401 Unauthorized

El endpoint de lectura queda así desacoplado del acceso directo a datos, debido a que el controlador utiliza las capas de DTO, servicio y repositorio.

5.2. Detalle del Endpoint de Escritura (POST) Refactorizado

Se implementó el endpoint:

POST /api/v1/contactos

Este endpoint permite registrar un nuevo contacto en la base de datos.

También requiere autenticación mediante:

Authorization: Bearer <token>

El cuerpo de la solicitud utiliza formato JSON.

Ejemplo para un contacto de tipo cliente:

```
{
  "tipo_contacto": "cliente",
  "nombre": "Carlos",
  "apellido": "Lopez",
  "correo": "carlos.api@example.com",
  "numero_telefonico": "55554321",
  "direccion": "Ciudad de Guatemala",
  "empresa": "Comercial API",
  "codigo_cliente": 2001
}
```

Los campos obligatorios generales son:

tipo_contacto
nombre
apellido
numero_telefonico

Dependiendo del tipo de contacto también existen campos obligatorios específicos:

Tipo	Campo adicional
general	No requiere campo específico
cliente	codigo_cliente
proveedor	descripcion
empleado	puesto y departamento

La solicitud sigue la siguiente arquitectura:

POST /api/v1/contactos

```

↓
Middleware JWT
↓
ContactoCreateRequest
↓
ContactoController@store
↓
ContactoCreateDTO
↓
ContactoService
↓
ContactoRepository
↓
DB::transaction()
↓
Eloquent
↓
MySQL

```

`ContactoCreateRequest` es responsable de validar la información recibida.

Después de superar la validación, el controlador crea un `ContactoCreateDTO` y obtiene el identificador del usuario directamente desde el token JWT.

Esto evita que el cliente pueda establecer manualmente el campo:

`id_usuario`

El servicio posteriormente solicita al repositorio que realice la creación.

En el repositorio se utiliza:

`DB::transaction()`

para asegurar que los datos generales y específicos se guarden de forma atómica.

Por ejemplo, al registrar un cliente se realizan dos operaciones:

```

INSERT contactos
↓
INSERT clientes

```

Si una de ellas falla, la transacción se revierte y no queda información incompleta en la base de datos.

Una creación exitosa responde:

201 Created

Ejemplo:

```
{
  "success": true,
  "message": "Contacto creado correctamente.",
  "data": {
    "id_contacto": 4,
    "id_usuario": 1,
    "tipo_contacto": "cliente",
    "nombre": "Carlos",
    "apellido": "Lopez",
    "correo": "carlos.api@example.com",
    "numero_telefonico": "55554321",
    "direccion": "Ciudad de Guatemala",
    "empresa": "Comercial API",
    "estado_activo": true,
    "cliente": {
      "id_contacto": 4,
      "codigo_cliente": 2001
    },
    "proveedor": null,
    "empleado": null
  }
}
```

También se comprobó el comportamiento ante datos inválidos.

Cuando la solicitud no cumple las reglas definidas, la API responde:

422 Unprocessable Entity

Ejemplo:

```
{
  "success": false,
  "message": "Los datos proporcionados no son válidos.",
  "errors": {
    "correo": [
      "El campo correo debe ser una dirección de correo válida."
    ],
    "numero_telefonico": [
      "El formato del campo numero telefonico no es válido."
    ],
    "codigo_cliente": [
      "El campo codigo cliente es obligatorio cuando tipo contacto es cliente."
    ]
  }
}
```

La validación se ejecuta antes de acceder al repositorio, evitando modificaciones innecesarias en la base de datos.

5.3. Evidencias de Ejecución (Postman/Insomnia)

Para comprobar el funcionamiento de los endpoints se realizaron pruebas utilizando **Postman**.

Se conservaron cuatro evidencias principales.

Evidencia 1 — Inicio de sesión

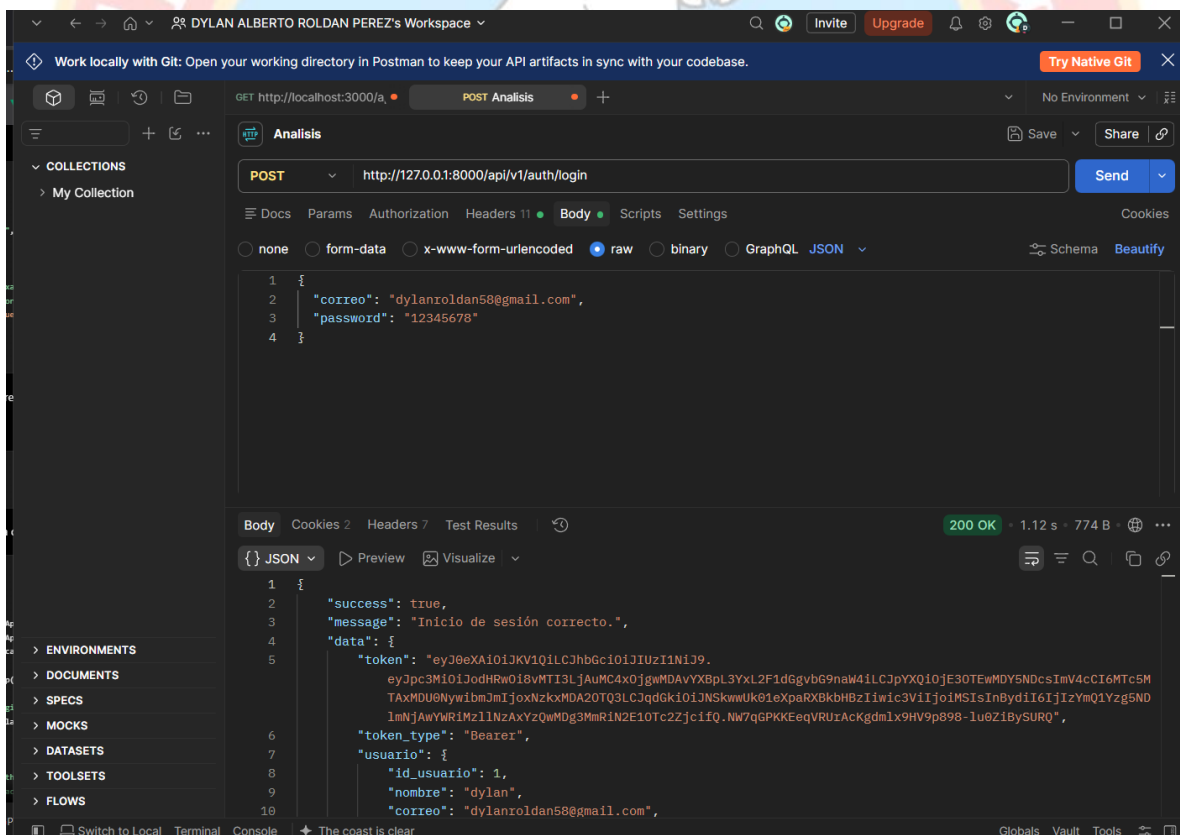
Petición:

POST /api/v1/auth/login

Resultado esperado y obtenido:

200 OK

La API devuelve un Bearer Token JWT que permite acceder a los endpoints protegidos.



Evidencia 2 — Consulta de contactos

Petición:

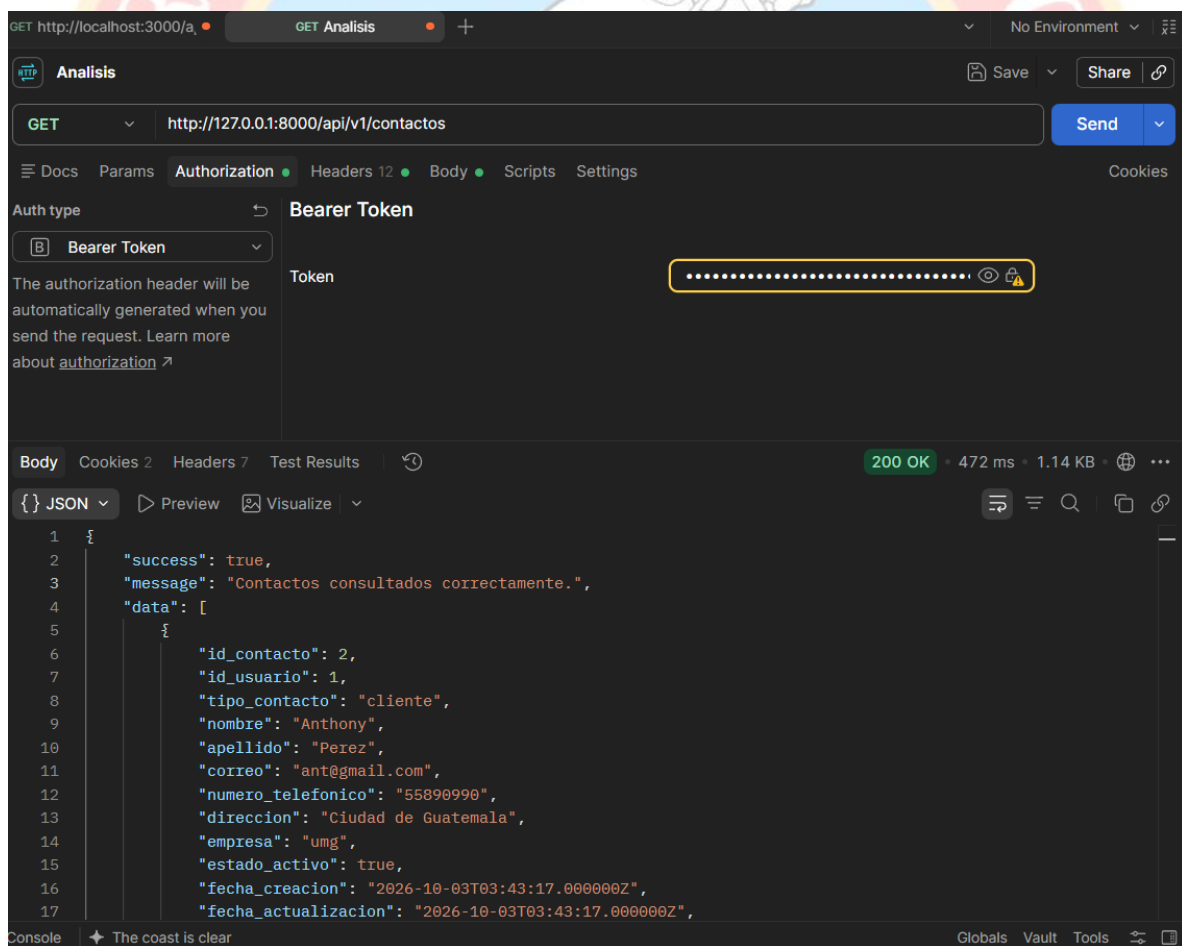
GET /api/v1/contactos

Se utilizó el token JWT obtenido previamente.

Resultado:

200 OK

La respuesta muestra los contactos obtenidos desde la base de datos junto con la información de paginación.



Evidencia 3 — Creación correcta de contacto

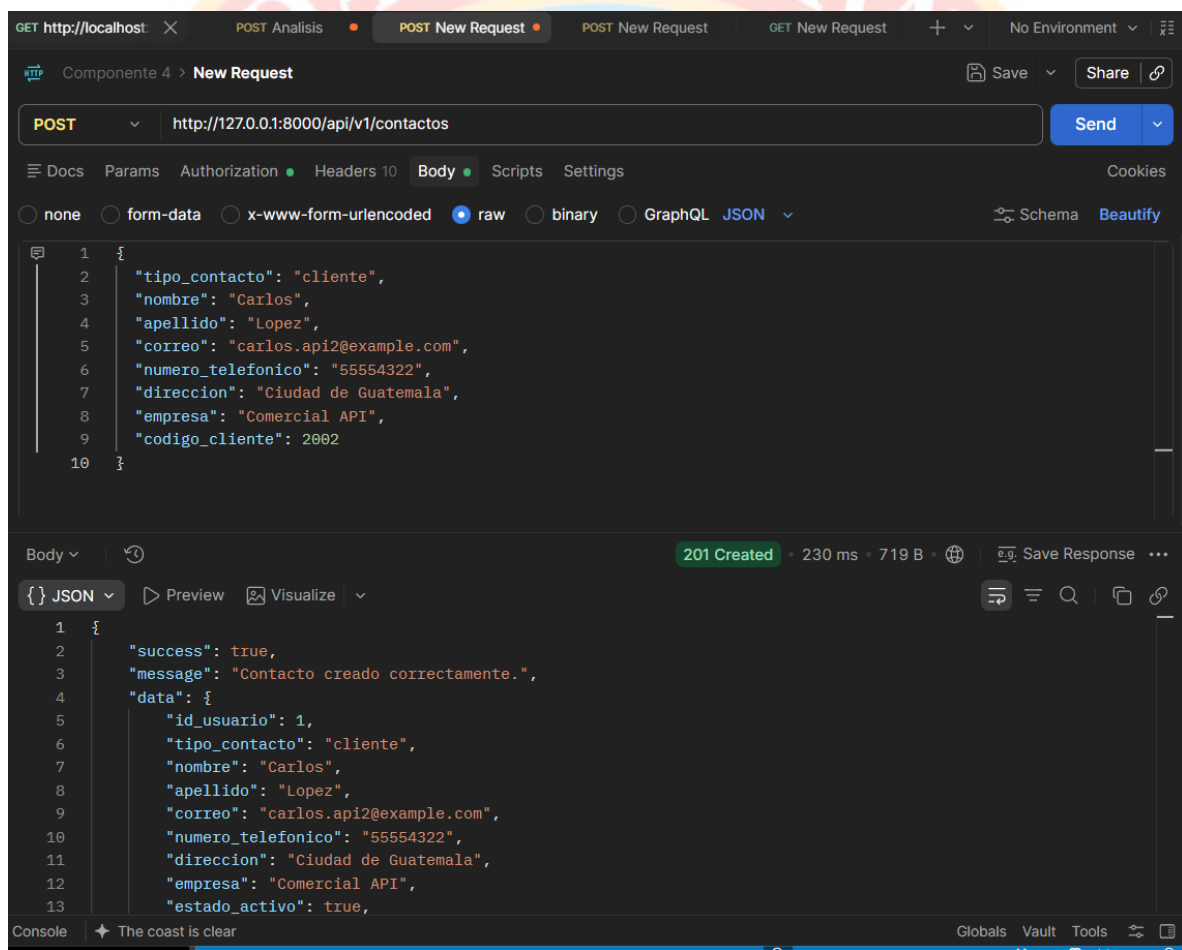
Petición:

POST /api/v1/contactos

Resultado:

201 Created

Se comprobó que el nuevo contacto fue almacenado correctamente en la base de datos.



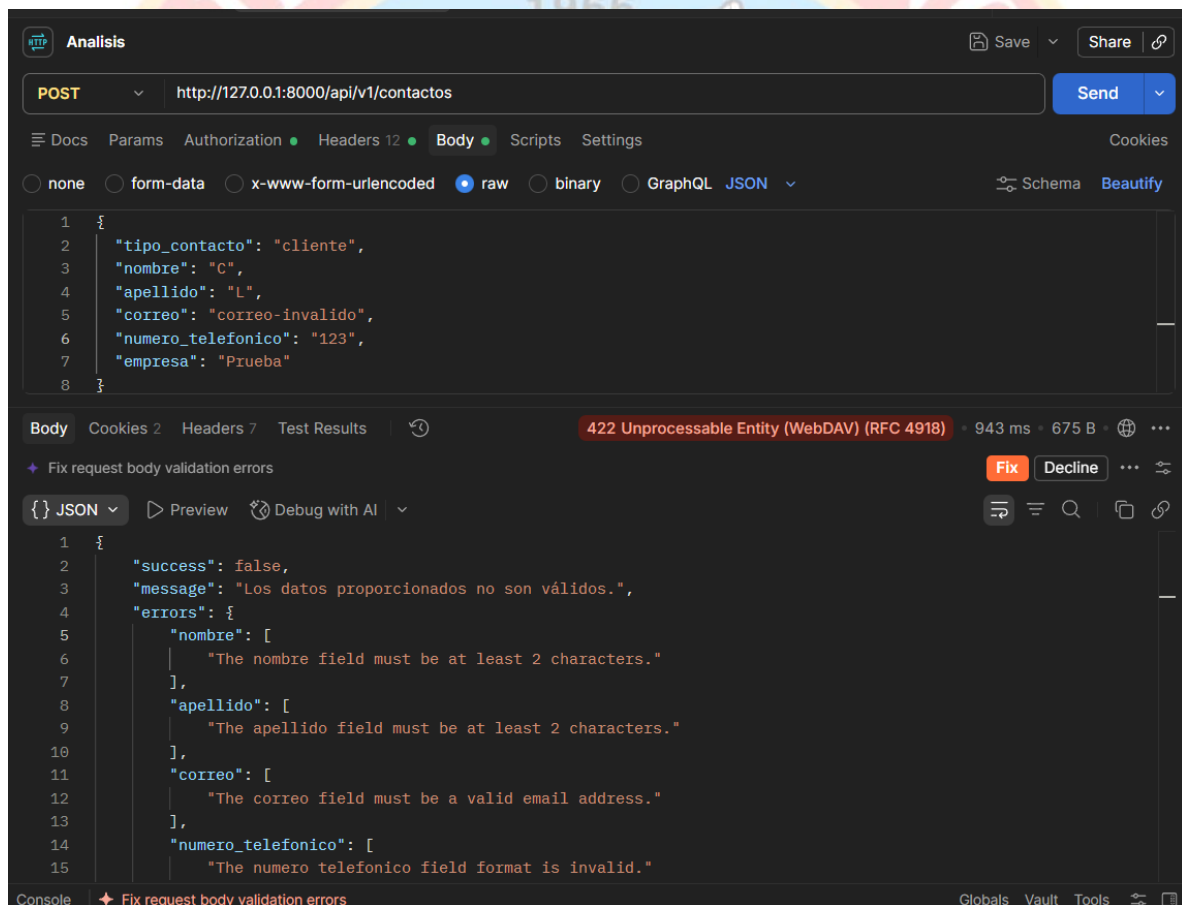
Evidencia 4 — Validación de datos incorrectos

Se envió una solicitud de creación con datos inválidos.

Resultado:

422 Unprocessable Entity

La API devolvió los errores de validación correspondientes sin insertar información inválida en la base de datos.



Colección de pruebas

Las solicitudes utilizadas durante las pruebas fueron agrupadas y exportadas desde Postman en formato Collection v2.1.

Vínculo a la colección de pruebas:

[COLECCIÓN POSTMAN](#)

6. MATRIZ DE TRAZABILIDAD ÁGIL

La matriz permite relacionar las historias de usuario definidas en el backlog con sus escenarios BDD, interfaces de usuario, contratos API y componentes del backend. De esta forma, se mantiene trazabilidad desde el requerimiento hasta su implementación técnica.

Requisito / HU	Archivo BDD (.feature)	Pantalla Figma	Endpoint API	Componente Backend
HU-01 – Iniciar sesión	autenticacion.feature	P01 - Login	POST /api/v1/auth/login	Api/V1/AuthController.php, User.php
HU-02 – Consultar contactos propios	gestion_contactos.feature	Contactos	GET /api/v1/contactos	ContactoController.php, ContactoService, ContactoRepository
HU-03 – Consultar todos los contactos como administrador	gestion_contactos.feature	Contactos	GET /api/v1/contactos	ContactoController.php, ContactoService, ContactoRepository
HU-04 – Registrar contacto general	gestion_contactos.feature	Nuevo contacto	POST /api/v1/contactos	ContactoController.php, Contacto.php, ContactoService
HU-05 – Registrar cliente	clasificacion_contactos.feature	Nuevo contacto	POST /api/v1/contactos	ContactoController.php, Contacto.php, Cliente.php
HU-06 – Registrar proveedor	clasificacion_contactos.feature	Nuevo contacto	POST /api/v1/contactos	ContactoController.php, Contacto.php, Proveedor.php

Requisito / HU	Archivo BDD (.feature)	Pantalla Figma	Endpoint API	Componente Backend
HU-07 – Registrar empleado	clasificacion_contactos.feature	Nuevo contacto	POST /api/v1/contactos	ContactoController.php, Contacto.php, Empleado.php
HU-08 – Buscar y filtrar contactos	gestion_contactos.feature	Contactos	GET /api/v1/contactos	ContactoController.php, ContactoFilterDTO, ContactoService, ContactoRepository

Observación de implementación

Dentro del primer incremento se logró realizar el acoplamiento inicial de la API con Laravel y la base de datos. Se implementó la estructura:

Controller → DTO → Service → Repository → Eloquent → MySQL

También se configuró autenticación mediante **JWT**, quedando disponible el endpoint:

POST /api/v1/auth/login

y se verificó el funcionamiento de:

GET /api/v1/contactos

para la consulta de contactos.

El endpoint:

POST /api/v1/contactos

forma parte del contrato definido para las historias **HU-04, HU-05, HU-06 y HU-07**, permitiendo posteriormente registrar el contacto general y asociarlo, según corresponda, con las entidades Cliente, Proveedor o Empleado.

8. CONCLUSIONES Y LECCIONES APRENDIDAS

Conclusión 1

La adopción de un enfoque ágil permitió transformar los requerimientos generales del módulo CRM en historias de usuario pequeñas y comprensibles. Esto facilita identificar

qué necesita cada tipo de usuario y organizar el desarrollo en incrementos que pueden ser implementados y verificados progresivamente.

Conclusión 2

El uso de criterios de aceptación y escenarios BDD permitió establecer una relación clara entre los requerimientos funcionales y el comportamiento esperado del sistema. Los archivos `.feature` sirven como evidencia de que cada historia de usuario posee condiciones específicas que posteriormente pueden convertirse en pruebas funcionales.

Conclusión 3

La elaboración del prototipo en Figma permitió validar de forma anticipada la interacción del usuario con funciones como inicio de sesión, consulta, búsqueda y registro de contactos. Esto reduce cambios posteriores en el desarrollo, ya que existe una referencia visual antes de implementar completamente las funcionalidades.

Conclusión 4

El acoplamiento inicial del backend demostró la utilidad de separar responsabilidades mediante controladores, DTO, servicios, repositorios y modelos. Esta estructura evita concentrar toda la lógica en los controladores y permite que el sistema sea más mantenible, escalable y fácil de probar.

Conclusión 5

La conexión entre backlog, BDD, diseño UX/UI, API y backend permitió mantener trazabilidad durante el desarrollo. Cada historia de usuario puede relacionarse con una pantalla, un escenario de comportamiento, un endpoint y uno o varios componentes del sistema, reduciendo el riesgo de implementar funciones que no correspondan con los requerimientos originales.

9. REFERENCIAS

1. Beck, K., et al. (2001). **Manifiesto for Agile Software Development**. Agile Alliance.
2. Schwaber, K., & Sutherland, J. (2020). **The Scrum Guide: The Definitive Guide to Scrum**.
3. Cucumber. **Gherkin Reference**. Documentación oficial de Cucumber.
4. Laravel. **Laravel Documentation**. Documentación oficial del framework Laravel.
5. OpenAPI Initiative. **OpenAPI Specification**. Especificación para el diseño y documentación de APIs REST.

6. Figma. **Figma Documentation**. Documentación para diseño de interfaces y prototipos interactivos.
7. RFC 7519. **JSON Web Token (JWT)**. Internet Engineering Task Force.
8. Atlassian. **Jira Software Documentation**. Documentación para administración de backlog, historias de usuario y metodologías ágiles.

